Stream API

Streams allow functional-style operations on collections and data sources. They are pipelines of computations that follow the pattern: **source -> zero or more intermediate ops -> terminal op**.

Stream API in Java 8 provides a functional approach to processing collections. It allows us to perform operations like filtering, mapping, reducing, and collecting in a declarative manner.

Key concepts

- **Source:** Collections, arrays, I/O channels, generator functions.
- **Intermediate operations:** `filter`, `map`, `flatMap`, `distinct`, `sorted`, `peek`, `limit`, `skip`. They are *lazy*.
- **Terminal operations:** `collect`, `forEach`, `reduce`, `count`, `anyMatch`, `allMatch`, `noneMatch`. They trigger execution.
- **Stateless vs stateful operations:** `filter` is stateless; `sorted` and `distinct` are stateful.
- **Ordered vs unordered streams:** Collections have encounter order; parallel pipelines and unordered() can change semantics/performance.
- Streams are not data structures; they operate on collections of data.
- Two types: **Intermediate Operations** (return another stream) and **Terminal Operations** (produce a result).
- Supports lazy evaluation.

Example: simple pipeline

```
List<Person> adults = people.stream()
    .filter(p -> p.getAge() >= 18)
    .sorted(Comparator.comparing(Person::getName))
    .collect(Collectors.toList());

List<String> names = Arrays.asList("Alice", "Bob", "Charlie");
List<String> result = names.stream()
    .filter(name -> name.startsWith("A"))
    .map(String::toUpperCase)
    .collect(Collectors.toList());
System.out.println(result); // [ALICE]
```

Parallel streams

Concept:
Parallel Streams in Java 8 enable multi-threaded data processing. Instead of processing data sequentially, the workload is split into multiple sub-tasks that run in parallel, taking advantage of multi-core processors.

Key Points:

1. **Creation:**
    - Convert an existing stream to parallel using `.parallel()`.
    - Directly create a parallel stream using `.parallelStream()` from a collection.
2. **When to Use:**
    - Ideal for large datasets and CPU-intensive operations.
    - Avoid for small datasets due to overhead of thread management.
3. **Threading Model:**
    - Uses the **ForkJoinPool** common pool by default.
    - Threads are automatically managed, but you can configure custom pools if needed.
4. **Order of Processing:**
    - Parallel streams may not preserve the encounter order unless explicitly required with `.forEachOrdered()`.
5. **Performance Considerations:**
    - Faster when tasks are **independent** and **stateless**.
    - Can be slower for I/O-bound or synchronized operations.

Example – Sequential vs Parallel:

```
List<Integer> numbers = IntStream.rangeClosed(1,
10).boxed().collect(Collectors.toList());

// Sequential processing
numbers.stream()
       .forEach(n -> System.out.println(Thread.currentThread().getName() +
" : " + n));
```

```
// Parallel processing
numbers.parallelStream()
       .forEach(n -> System.out.println(Thread.currentThread().getName() +
" : " + n));
```

Output will show multiple threads when using `parallelStream()`.

Example – Using forEachOrdered to Maintain Order:

```
numbers.parallelStream()
       .forEachOrdered(n -> System.out.print(n + " "));
```

Output:

```
1 2 3 4 5 6 7 8 9 10
```

`Tips:`

- Use `.parallelStream()` or `.parallel()` to process elements in parallel.
- Avoid shared mutable state; prefer immutable state.

Best Practices:

- Use for CPU-intensive, non-blocking tasks.
- Avoid shared mutable state (to prevent concurrency issues).
- Measure performance before deciding to switch to parallel streams.

ForEach() Method

Concept: The `forEach()` method is used to iterate over elements in a collection or stream.

Key Points:

- It is a terminal operation in the Stream API.
- Accepts a `Consumer` functional interface.

Example:

```
List<String> list = Arrays.asList("Apple", "Banana", "Cherry");
list.forEach(item -> System.out.println(item));
list.stream().forEach(System.out::println);
```